Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

# ОТЧЁТ
по лабораторной работе №8
по дисциплине
«Информационные технологии и веб-программирование»
Тема: Контейнеризация приложения с Docker

Выполнила: студентка группы 320604
Хаджинова К.А.

Проверил: Хаджинова Н.В.

Минск 2026

**СОДЕРЖАНИЕ**

1 ЦЕЛЬ РАБОТЫ 3
2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 3
3 ХОД РАБОТЫ 3
3.1 Состав приложения 3
3.2 Dockerfile сервера 4
3.3 Dockerfile клиента 4
3.4 Файл docker-compose.yml 4
3.5 Дополнительные требования 6
3.6 Проверка работы 8
ЗАКЛЮЧЕНИЕ 9
ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ 10
ПРИЛОЖЕНИЕ А Листинг файла Dockerfile сервера 11
ПРИЛОЖЕНИЕ Б Листинг файла Dockerfile клиента 12
ПРИЛОЖЕНИЕ В Листинг файла docker-compose.yml 13
ПРИЛОЖЕНИЕ Г Листинг файла nginx.conf 16
ПРИЛОЖЕНИЕ Д Образец файла .env.example 18
ПРИЛОЖЕНИЕ Е Листинг файла app.js сервера с маршрутом /health 19

# 1 ЦЕЛЬ РАБОТЫ

Освоить контейнеризацию: описать образы сервера и клиента файлами Dockerfile, собрать их и запустить вместе с базой данных одной командой Docker Compose. Упаковывается DSLKit, веб-редактор диаграмм как кода из направления SaaS, то есть сервер с аутентификацией из лабораторной работы 3, клиент на React из лабораторной работы 5 и база PostgreSQL. Это основа развёртывания будущего курсового проекта.

# 2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

**Образ и контейнер.** Образ состоит из неизменяемых слоёв файловой системы и описания запуска. Контейнер является запущенным экземпляром образа с тонким слоем для записи, поэтому при удалении контейнера всё записанное в нём пропадает.

**Dockerfile.** Файл перечисляет инструкции сборки: `FROM` выбирает базовый образ, `WORKDIR` задаёт папку, `COPY` переносит файлы, `RUN` выполняет команду при сборке, `EXPOSE` документирует порт, `CMD` задаёт команду запуска. Каждая инструкция создаёт слой, а неизменившиеся слои берутся из кэша, поэтому файлы зависимостей копируют раньше исходного кода.

**Многоэтапная сборка.** В одном Dockerfile несколько инструкций `FROM`. Первый этап собирает проект со всеми инструментами, а в конечный образ инструкция `COPY --from=builder` переносит только результат. Node.js и исходный код клиента в итоговый образ не попадают.

**Сети и тома.** Контейнеры одной сети Compose находят друг друга по имени сервиса. Том хранится вне контейнера и сохраняет данные базы при его пересоздании.

**Docker Compose.** Файл `docker-compose.yml` описывает сервисы, сети, тома и порядок запуска. Команда `docker compose up --build` собирает образы и запускает всё приложение.

# 3 ХОД РАБОТЫ

## 3.1 Состав приложения

Приложение состоит из трёх сервисов. Сервис `db` запускает PostgreSQL 16, сервис `backend` собирается из сервера лабораторной работы 3, сервис `frontend` собирается из клиента лабораторной работы 5. Код приложений не копировался: Compose указывает на их папки как на контекст сборки, а файлы Docker лежат в папке `part-2/lab-08-docker`. Приложение запускается командой `docker compose up --build`, а перед этим нужно

скопировать `.env.example` в `.env` и заполнить значения. Образы собираются без ошибок, а контейнеры работают, как показано на рисунке 1.

```
$ docker ps --format "table {{.Names}} {{.Image}} {{.Status}} {{.Ports}}"
NAMES              IMAGE               STATUS                          PORTS
dslkit-frontend-1  dslkit-frontend      Up 1 second (health: starting)  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
dslkit-backend-1   dslkit-backend       Up 7 seconds (healthy)          127.0.0.1:3000->3000/tcp
dslkit-db-1        postgres:16-alpine  Up 11 seconds (healthy)         5432/tcp
```

Рисунок 1 – Три работающих контейнера со статусом healthy, команда docker ps

### 3.2 Dockerfile сервера

Образ сервера собирается в два этапа, листинг приведён в приложении А. На первом этапе команда `npm ci --omit=dev` ставит только рабочие зависимости. Нативные модули bcrypt и sqlite3 берутся готовыми для Alpine, поэтому компилятор не нужен. Второй этап копирует папку `node_modules` и исходный код в чистый образ без кэша npm. Файл `Dockerfile.dockerignore` исключает тесты, отчёты, `.env` и локальные базы, чтобы секреты и лишнее не попали в образ.

Процесс работает от пользователя `node`, а не от root. При запуске выполняются миграции Sequelize, затем идемпотентный сидер администратора, который пропускается без переменных `ADMIN_EMAIL` и `ADMIN_PASSWORD`, и после этого стартует сервер. Таблицы в пустой базе появляются сами.

### 3.3 Dockerfile клиента

Образ клиента собирается в два этапа, листинг приведён в приложении Б. Этап `builder` ставит зависимости и выполняет `npm run build`. Адрес сервера при сборке задаётся значением `/api`, поэтому браузер обращается к тому же адресу, с которого открыта страница, и CORS не нужен. Конечный образ построен на `nginx-unprivileged`: в нём только Nginx и папка `dist`, а процесс работает от обычного пользователя на порту 8080. Размер образа клиента около 74 МБ, образа сервера около 313 МБ.

### 3.4 Файл docker-compose.yml

Листинг приведён в приложении В. Связь между контейнерами идёт по имени сервиса: сервер подключается к базе по адресу `db:5432`, а Nginx

пересылает запросы на `backend:3000`. База хранит данные в именованном томе `pgdata`, поэтому пересоздание контейнера их не стирает. Это проверено: после `docker compose restart db` пользователи остались в таблице. Порядок запуска задан условиями `service_healthy`: сервер ждёт готовности базы, а клиент ждёт готовности сервера.

Приложение открывается по адресу `http://localhost:8080`. Порт 80 не использован, потому что он часто занят на рабочих машинах, и значение меняется переменной `FRONTEND_PORT`. В тексте задания названы порты 80 и 5000, их дают значения `FRONTEND_PORT=80` и `BACKEND_PORT=5000` в `.env`. Стек останавливается командой `docker compose down`, до этого контейнеры видны в `docker ps`, а `docker compose down -v` дополнительно удаляет том `pgdata` с данными базы. Результат показан на рисунке 2.

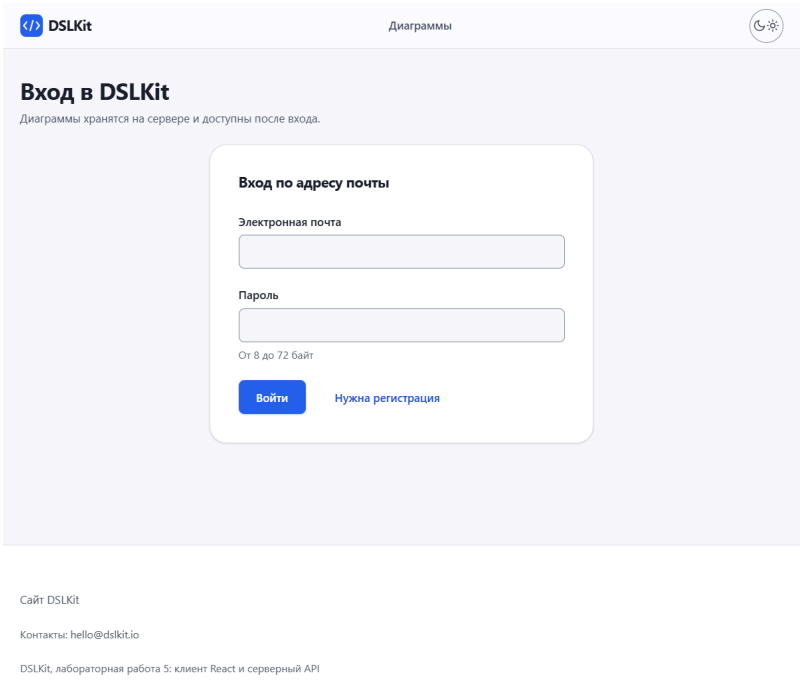



Рисунок 2 – Клиент из контейнера Nginx по адресу localhost:8080

Для проверки работы целиком создан пользователь и три диаграммы через форму и API, а список показан на рисунке 3. Запросы идут через прокси Nginx в контейнер сервера, а оттуда в базу PostgreSQL.

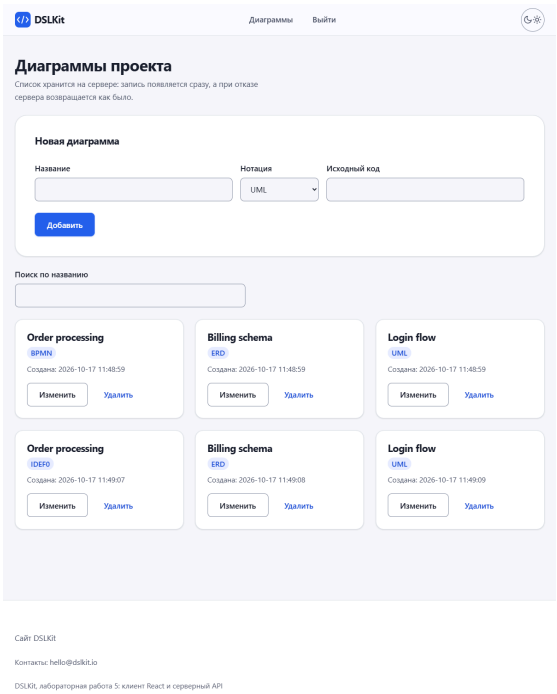



Рисунок 3 – Список диаграмм, полученный из PostgreSQL через Nginx и сервер

### 3.5 Дополнительные требования

Выполнено восемь дополнительных требований из списка условия и несколько мер безопасности.

1. **Переменные окружения.** Строка подключения, пароль базы и секрет JWT берутся из файла `.env`. Файл не попадает в репозиторий, а образец `.env.example` не содержит значений. Без `POSTGRES_PASSWORD` и `JWT_SECRET` Compose отказывается запускаться и называет недостающую переменную (приложение Д).
2. **Healthcheck сервера.** В сервер добавлен маршрут `GET /health`, который проверяет соединение с базой и отвечает 200 или 503. Маршрут стоит раньше журнала запросов, поэтому проверки не заполняют таблицу журнала. Код показан в приложении Е, а проверяют его два теста.
3. **Healthcheck базы и `service_healthy`.** Команда `pg_isready` проверяет готовность PostgreSQL, и сервер запускается только после неё.
4. **Изоляция сетей.** Сеть `internal` объявлена с признаком `internal: true`: у неё нет выхода наружу, и в ней только база и сервер. Сеть `public` соединяет сервер и клиент. Порт базы на хост не публикуется.
5. **Reverse proxy на Nginx.** Nginx отдаёт статические файлы, возвращает `index.html` для маршрутов одностраничного приложения и пересылает

`/api/` на сервер (приложение Г). Передаётся `X-Forwarded-For`, а в сервере включено `TRUST_PROXY=1`, поэтому ограничитель запросов видит настоящий адрес клиента.

6. **Запуск не от root.** Сервер работает от `node`, Nginx от `nginx`. Контейнер сервера дополнительно запущен с файловой системой только для чтения, без привилегий (`cap_drop: ALL`, `no-new-privileges`).
7. **Файл `.dockerignore`.** Исключает `node_modules`, тесты, отчёты и `.env` из контекста сборки.
8. **Зеркало реестра.** Для Республики Беларусь предусмотрена переменная `REGISTRY`: значение `mirror.gcr.io/` ставится перед именами базовых образов в обоих Dockerfile и в Compose. Сборка и загрузка образа PostgreSQL через это зеркало проверены, а в основном запуске образы скачивались с Docker Hub напрямую. Вместо переменной зеркало можно задать для всего демона Docker в `daemon.json` ключом `"registry-mirrors": ["https://mirror.gcr.io", "https://dockerhub.timeweb.cloud"]`. При последней правке отчёта демон Docker на компьютере автора не работал, поэтому стек повторно не запускался, а рисунки получены при прежнем запуске.
9. **Лимиты ресурсов и ротация журналов.** У каждого сервиса заданы предел памяти (`mem_limit`), предел числа процессов (`pids_limit`) и ротация журналов контейнера (`max-size: 10m`, три файла). Один сбойный сервис не займёт всю память и диск компьютера.
10. **Необязательные pgAdmin и Redis.** Оба сервиса описаны в `docker-compose.yml` под профилями `tools` и `cache`, поэтому обычный запуск их не поднимает. Команда `docker compose --profile tools up -d pgadmin` запускает pgAdmin на адресе `127.0.0.1:5050`, доступном только с этого компьютера. Redis стоит только во внутренней сети и портов наружу не открывает.

Состояние стенда показано на рисунке 4: ответы `/health` через прокси и напрямую, сети и том проекта.

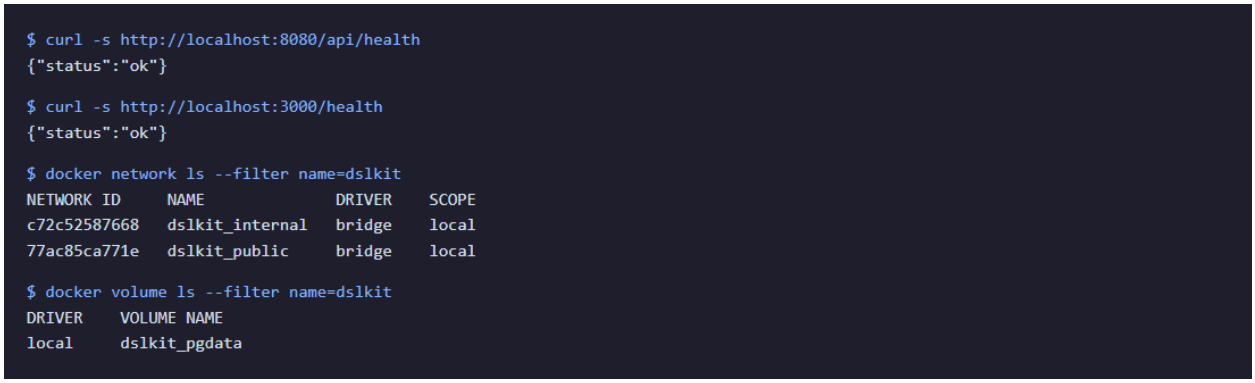



Рисунок 4 – Ответы /health, внутренняя и внешняя сети и том pgdata

### 3.6 Проверка работы

Структура файлов Docker проверена двадцатью одним автоматическим тестом, которые работают без запущенного Docker. Они проверяют двухэтапную сборку клиента, установку только рабочих зависимостей, пользователя `node`, условия `service_healthy`, том базы, закрытую сеть, отсутствие значений секретов в образце и прокси Nginx. Двенадцать из них проверяют защиту: версии образов без тега `latest`, отсутствие паролей в образах, закрытые порты и сети, снятые права контейнеров и заголовки Nginx. Сервер проверен 137 тестами, включая два теста маршрута `/health`. Запуск описан в `README.md`. Ветка репозитория: 28.

# ЗАКЛЮЧЕНИЕ

Сервер, клиент и база данных упакованы в контейнеры и запускаются одной командой. Клиент собирается в два этапа и отдаётся через Nginx, и тот же Nginx служит прокси для сервера. Данные базы сохраняются в томе, а порядок запуска определяют проверки здоровья. Секреты вынесены в `.env`, сети разделены, процессы работают без прав root. Такая основа подходит для развёртывания курсового проекта.

## ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

9. **Как в Docker Compose задать зависимость запуска контейнеров (чтобы база данных стартовала раньше backend)?** Ключом `depends_on`. Чтобы сервер ждал именно готовности базы, а не только старта контейнера, добавляют `condition: service_healthy` и healthcheck у базы.
10. **Как выполнить команду внутри запущенного контейнера (например, зайти в оболочку)?** Командой `docker exec -it <контейнер> sh`, а в Compose `docker compose exec backend sh`.
11. **Как пробросить порт с хоста в контейнер в Docker Compose?** Ключом `ports` в формате `"хост:контейнер"`, например `"8080:8080"`. Можно привязать к адресу, как `127.0.0.1:3000:3000`.

## ПРИЛОЖЕНИЕ А

(обязательное)

## Листинг файла Dockerfile сервера

```
# Build context: ../lab-03-auth (set in docker-compose.yml).
# REGISTRY is an optional prefix of a mirror, for example mirro
r.gcr.io/ (note the trailing slash).
ARG REGISTRY=

# Stage 1: production dependencies only, native modules (bcrypt,
sqlite3) come as prebuilt binaries for Alpine.
FROM ${REGISTRY}library/node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force

# Stage 2: small runtime image without npm cache, tests and repo
rts (see Dockerfile.dockerignore).
FROM ${REGISTRY}library/node:22-alpine
WORKDIR /app
ENV PORT=3000
COPY --from=deps --chown=node:node /app/node_modules ./node_modu
les
COPY --chown=node:node . .
# The official image ships a non-root user "node": the server ne
ver runs as root.
USER node
EXPOSE 3000
HEALTHCHECK --interval=15s --timeout=3s --start-period=20s --ret
ries=3 \
  CMD node -e "fetch('http://127.0.0.1:'+process.env.PORT+'/heal
th').then(r=>process.exit(r.ok?0:1),()=>process.exit(1))"
# Tables are created by the migrations, the admin seed is idempo
tent and skipped without ADMIN_EMAIL.
CMD ["sh", "-c", "npx sequelize-cli db:migrate && npx sequelize-
cli db:seed:all && exec node server.js"]
```

## ПРИЛОЖЕНИЕ Б

(обязательное)

## Листинг файла Dockerfile клиента

```
# Build context: ../lab-05-react-api, extra context "deploy" is
this folder (set in docker-compose.yml).
ARG REGISTRY=

# Stage 1: build the static files with Vite.
FROM ${REGISTRY}library/node:22-alpine AS builder
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
# The browser calls /api, Nginx forwards it to the backend, so n
o CORS and no host name is baked into the bundle.
ARG VITE_API_URL=/api
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

# Stage 2: only Nginx and the built files reach the final image,
Node.js and sources stay in the builder.
FROM ${REGISTRY}nginxinc/nginx-unprivileged:1.27-alpine
COPY --from=deploy nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=builder /app/dist /usr/share/nginx/html
# The unprivileged image runs as a non-root user and listens on
8080.
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=3s --start-period=5s --retr
ies=3 \
  CMD wget -q -O /dev/null http://127.0.0.1:8080/ || exit 1
CMD ["nginx", "-g", "daemon off;"]
```

## ПРИЛОЖЕНИЕ В

(обязательное)

**Листинг файла docker-compose.yml**

```
# DSLKit: React client (Nginx) + Express API + PostgreSQL. Star
t: docker compose up --build
name: dslkit

# Container logs rotate, so a long run cannot fill the disk of t
he host.
x-logging: &logging
  driver: json-file
  options:
    max-size: 10m
    max-file: '3'

services:
  db:
    image: ${REGISTRY:-}library/postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-dslkit}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set POSTGRES_PASSW
ORD in .env}
      POSTGRES_DB: ${POSTGRES_DB:-dslkit}
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks: [internal]
    logging: *logging
    mem_limit: 512m
    pids_limit: 200
    healthcheck:
      test: ['CMD-SHELL', 'pg_isready -U $${POSTGRES_USER} -d
$${POSTGRES_DB}']
      interval: 5s
      timeout: 3s
      retries: 10

  backend:
    build:
      context: ../lab-03-auth
      dockerfile: ../lab-08-docker/backend/Dockerfile
      args:
        REGISTRY: ${REGISTRY:-}
    restart: unless-stopped
    environment:
      PORT: 3000
      DATABASE_URL: postgres://${POSTGRES_USER:-dslkit}:${POSTGR
ES_PASSWORD}@db:5432/${POSTGRES_DB:-dslkit}
      JWT_SECRET: ${JWT_SECRET:?set JWT_SECRET in .env, at least
```

```
32 random characters}
      JWT_EXPIRES_IN: ${JWT_EXPIRES_IN:-1h}
      ADMIN_EMAIL: ${ADMIN_EMAIL:-}
      ADMIN_PASSWORD: ${ADMIN_PASSWORD:-}
      # Exactly one reverse proxy (Nginx) stands in front, so th
e rate limit sees the real client address.
      TRUST_PROXY: '1'
    depends_on:
      db:
        condition: service_healthy
    ports:
      # Published for checks from the host only, the browser use
s the proxy /api of the frontend.
      - '127.0.0.1:${BACKEND_PORT:-3000}:3000'
    read_only: true
    tmpfs: [/tmp]
    cap_drop: [ALL]
    security_opt: ['no-new-privileges:true']
    networks: [internal, public]
    logging: *logging
    mem_limit: 512m
    pids_limit: 200

  frontend:
    build:
      context: ../lab-05-react-api
      dockerfile: ../lab-08-docker/frontend/Dockerfile
      additional_contexts:
        deploy: ./frontend
      args:
        REGISTRY: ${REGISTRY:-}
    restart: unless-stopped
    depends_on:
      backend:
        condition: service_healthy
    ports:
      - '${FRONTEND_PORT:-8080}:8080'
    # nginx-unprivileged runs as a normal user on port 8080, so
it needs no capability and no way to gain one.
    cap_drop: [ALL]
    security_opt: ['no-new-privileges:true']
    networks: [public]
    logging: *logging
    mem_limit: 128m
    pids_limit: 100

  # Optional tools, off by default: docker compose --profile too
ls up -d pgadmin
  pgadmin:
    profiles: [tools]
    image: ${REGISTRY:-}dpage/pgadmin4:8
    restart: unless-stopped
```

```
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_EMAIL:-admin@example.com}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_PASSWORD:-}
      PGADMIN_CONFIG_SERVER_MODE: 'False'
      PGADMIN_CONFIG_MASTER_PASSWORD_REQUIRED: 'False'
    depends_on:
      db:
        condition: service_healthy
    ports:
      # Only this computer can open it, it is a tool for the dev
eloper.
      - '127.0.0.1:${PGADMIN_PORT:-5050}:80'
    networks: [internal, public]
    logging: *logging
    mem_limit: 512m
    pids_limit: 200

  # Optional cache, off by default: docker compose --profile cac
he up -d redis. The server does not use it yet.
  redis:
    profiles: [cache]
    image: ${REGISTRY:-}library/redis:7-alpine
    restart: unless-stopped
    command: ['redis-server', '--save', '', '--appendonly', 'n
o']
    networks: [internal]
    healthcheck:
      test: ['CMD', 'redis-cli', 'ping']
      interval: 5s
      timeout: 3s
      retries: 10
    logging: *logging
    mem_limit: 128m
    pids_limit: 100

networks:
  # No access from outside: only the backend reaches the databas
e.
  internal:
    internal: true
  public:

volumes:
  pgdata:
```

## ПРИЛОЖЕНИЕ Г
(обязательное)
**Листинг файла nginx.conf**

```
limit_req_zone $binary_remote_addr zone=api:10m rate=10r/s;

server {
  listen 8080;
  server_name _;
  server_tokens off;
  # The API only takes small JSON bodies, and one address cannot
flood it.
  client_max_body_size 100k;
  root /usr/share/nginx/html;
  index index.html;

  add_header X-Content-Type-Options "nosniff" always;
  add_header X-Frame-Options "DENY" always;
  add_header Referrer-Policy "no-referrer" always;
  add_header Content-Security-Policy "default-src 'self'; script
-src 'self'; style-src 'self'; img-src 'self' data:; connect-src
'self'; object-src 'none'; base-uri 'none'; frame-ancestors 'non
e'; form-action 'self'" always;

  # Reverse proxy: /api/health becomes /health on the backend se
rvice, found by its name in the compose network.
  location /api/ {
    proxy_pass http://backend:3000/;
    limit_req zone=api burst=20 nodelay;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 15s;
  }

  # Hashed build files can be cached for a long time.
  # A location with its own add_header does not inherit the head
ers of the server block, so all of them are repeated here.
  location /assets/ {
    expires 1y;
    add_header Cache-Control "public, immutable";
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;
    add_header Referrer-Policy "no-referrer" always;
    add_header Content-Security-Policy "default-src 'self'; scri
pt-src 'self'; style-src 'self'; img-src 'self' data:; connect-s
rc 'self'; object-src 'none'; base-uri 'none'; frame-ancestors
'none'; form-action 'self'" always;
  }
```

```
  # Single-page application: unknown paths return index.html.
  location / {
    try_files $uri /index.html;
  }
}
```

## ПРИЛОЖЕНИЕ Д
(обязательное)
## Образец файла .env.example

```
# Copy to .env and fill in. The real .env is never committed.
# Ready-made values are NOT provided on purpose.

# Optional mirror prefix for images when Docker Hub is blocked,
with a trailing slash. Empty: Docker Hub.
# REGISTRY=mirror.gcr.io/
REGISTRY=

POSTGRES_USER=dslkit
POSTGRES_DB=dslkit
# Letters and digits only (the value goes into a connection UR
L).
POSTGRES_PASSWORD=

# At least 32 random characters.
# Generate: node -e "console.log(require('crypto').randomBytes(4
8).toString('hex'))"
JWT_SECRET=
JWT_EXPIRES_IN=1h

# First administrator (optional): both values or none, the passw
ord needs 10 characters or more.
ADMIN_EMAIL=
ADMIN_PASSWORD=

# Ports on the host.
FRONTEND_PORT=8080
BACKEND_PORT=3000

# Optional pgAdmin (docker compose --profile tools up -d pgadmi
n). Open http://localhost:5050, the server name is db.
# The password is required for the tool to start, letters and di
gits.
PGADMIN_EMAIL=admin@example.com
PGADMIN_PASSWORD=
PGADMIN_PORT=5050
```

## ПРИЛОЖЕНИЕ Е

(обязательное)

### Листинг файла app.js сервера с маршрутом /health

```
const express = require('express');
const { createAuthenticate } = require('./middleware/auth');
const { createAuthController } = require('./controllers/auth.con
troller');
const { createUsersController } = require('./controllers/users.c
ontroller');
const { createDiagramsController } = require('./controllers/diag
rams.controller');
const { createAuthRouter } = require('./routes/auth.routes');
const { createUsersRouter } = require('./routes/users.routes');
const { createDiagramsRouter } = require('./routes/diagrams.rout
es');
const { securityHeaders, requireJson } = require('./middleware/h
ttp');
const { createRequestLogger } = require('./middleware/request-lo
g');
const { createRateLimiter, createLoginGuard } = require('./middl
eware/rate-limit');
const { routeNotFound, errorHandler } = require('./middleware/er
ror-handler');
const { createCors } = require('./middleware/cors');
const { checkJwtSettings } = require('./settings');
const LIMITS = require('../limits.json');

const MINUTE = 60 * 1000;
const DEFAULTS = {
  jwtExpiresIn: '1h',
  bcryptRounds: 10,
  // Origin of the React client of lab 5 that may call the API.
  corsOrigin: 'http://localhost:5178',
  // Failed logins per email before it is locked, and for how lo
ng.
  loginMaxAttempts: LIMITS.loginMaxAttempts,
  loginLockMs: LIMITS.loginLockMinutes * MINUTE,
  // Requests per client address in a window: login and registra
tion are limited separately.
  loginRateMax: LIMITS.loginRateMax,
  loginRateWindowMs: LIMITS.loginRateWindowMinutes * MINUTE,
  registerRateMax: LIMITS.registerRateMax,
  registerRateWindowMs: LIMITS.registerRateWindowMinutes * MINUT
E,
};

// settings: { jwtSecret, jwtExpiresIn?, bcryptRounds?, login*/r
egister* limits?, trustProxy?, corsOrigin? }
const createApp = (models, settings) => {
```

```
  const config = { ...DEFAULTS, ...settings };
  checkJwtSettings(config);
  const authenticate = createAuthenticate(models, config);
  const loginGuard = createLoginGuard({ maxAttempts: config.logi
nMaxAttempts, lockMs: config.loginLockMs });
  const limiters = {
    loginLimiter: createRateLimiter({ windowMs: config.loginRate
WindowMs, max: config.loginRateMax }),
    registerLimiter: createRateLimiter({ windowMs: config.regist
erRateWindowMs, max: config.registerRateMax }),
  };
  const app = express();
  app.disable('x-powered-by');
  app.use(createCors(config.corsOrigin));
  // Behind a reverse proxy req.ip is the proxy unless this is s
et (TRUST_PROXY), and the limits would count everyone as one cli
ent.
  if (config.trustProxy !== undefined) app.set('trust proxy', co
nfig.trustProxy);
  app.use(securityHeaders);
  // Liveness and database check for the Docker healthcheck, pla
ced before the request log so probes do not fill it.
  app.get('/health', async (req, res) => {
    try {
      await models.sequelize.authenticate();
      res.json({ status: 'ok' });
    } catch {
      res.status(503).json({ status: 'unavailable' });
    }
  });
  app.use(createRequestLogger(models));
  // The limit is set explicitly (limits.json), so it does not c
hange with the library default.
  app.use(express.json({ limit: LIMITS.bodyBytes }));
  app.use(requireJson);
  app.use('/auth', createAuthRouter(createAuthController(models,
config, loginGuard), limiters));
  app.use('/diagrams', createDiagramsRouter(createDiagramsContro
ller(models), authenticate));
  app.use(createUsersRouter(createUsersController(models), authe
nticate));
  app.use(routeNotFound);
  app.use(errorHandler);
  return app;
};

module.exports = { createApp };
```